

INSTITUTO TÉCNICO SALESIANO VILLADA
ESPECIALIDAD EN INFORMÁTICA



Apunte de Git
CONTROL DE VERSIONES

DOCENTES : Cortesini Perez Luciano Tomas.
Marchisone Mateo.
CÁTEDRA : Programación I.
CURSO : 4to año C

CÓRDOBA, ARGENTINA
6 de mayo de 2026

Índice general

1. Introducción	1
1.1. Que es Git?	1
1.2. Que es GitHub?	1
1.3. Por qué usamos Git?	1
2. Conceptos Básicos	2
2.1. Repositorio	2
2.2. Commit	2
2.3. Áreas de un proyecto Git	3
2.4. Branch	3
2.5. Puntero HEAD	4
2.6. Merge	5
2.7. Push y Pull	6
2.8. Pull Request	7
3. Usando Git en la práctica	8
3.1. Instalación	8
3.2. Configuración inicial	8
3.3. Como obtener ayuda	8
3.4. Creando un repositorio vacio	9
3.5. Realizando mi primer commit	9
3.6. Subir repositorio a GitHub	10
3.7. workflow	11

Introducción

1.1. Que es Git?

GIT (Global Information Tracker) es un **sistema de control de versiones distribuido**, gratuito y de código abierto, diseñado para **gestionar el historial de cambios** en el código fuente de proyectos de software de forma rápida y eficiente. Permite a los desarrolladores rastrear modificaciones, volver a versiones anteriores y trabajar colaborativamente sin sobrescribir el trabajo de otros.

Cuando decimos que Git es un sistema de control de versiones **distribuido**, significa que cada copia del repositorio **contiene el historial completo** del proyecto. No dependemos de un único servidor para trabajar: podemos hacer commits, crear ramas y consultar historial **sin conexión**. Si un servidor remoto falla, cualquier copia del repositorio sirve para recuperar la totalidad del contenido.

1.2. Que es GitHub?

GitHub es una **plataforma en línea** para alojar repositorios Git y **colaborar con otras personas**. Permite compartir código, revisar cambios, trabajar en equipo mediante ramas y pull requests, y mantener un respaldo remoto del proyecto.

Es importante distinguirlos: **Git no es GitHub**. Git es la herramienta que funciona en nuestra computadora (local), mientras que GitHub es un servicio web que usa Git para facilitar la colaboración y la publicación de repositorios. **Se puede usar Git sin GitHub**, y también existen otras plataformas similares (por ejemplo, GitLab o Bitbucket).

1.3. Por qué usamos Git?

Usar Git y GitHub mejora el flujo de trabajo porque permite:

- Guardar el historial completo de cambios: sabés **qué cambió, cuándo y quién lo cambió**.
- Podés **volver a cualquier versión anterior** si algo se rompe o si querés revisar cómo era el código antes de un cambio específico.
- Permite **colaborar de forma ordenada**, integrando aportes de varios desarrolladores sin pisar el trabajo de otros.
- Facilita la **revisión de código**, ya que podés **comparar cambios** y discutirlos antes de integrarlos al proyecto.
- Respaldo remoto: si tu computadora falla, tu trabajo no se pierde porque está guardado en GitHub.

Conceptos Básicos

2.1. Repositorio

Un repositorio en Git es un directorio que contiene tu proyecto junto con toda su historia: cada cambio que se hizo, cuándo, quién lo hizo y por qué. Técnicamente, es una carpeta normal con un subdirectorio oculto llamado `.git` donde Git guarda toda esa información. Ese `.git` es el corazón del repositorio: sin él, sería solo una carpeta común. Hay dos tipos principales:

Local — vive en tu máquina. Ahí trabajas, haces commits, creas ramas. Es completamente funcional sin internet.

Remoto — vive en un servidor (GitHub, GitLab, etc.). Sirve como punto central para compartir el trabajo con otros o como respaldo.

2.2. Commit

Un commit es una instantánea (foto) del estado de tu proyecto en un momento dado, guardada permanentemente en la historia del repositorio. Cada vez que haces un commit, Git toma todos los cambios que preparaste (los que están en el staging area) y los congela en un paquete inmutable que incluye:

- **El contenido** — cómo quedaron los archivos en ese momento.
- **Metadatos** — autor, fecha y hora.
- **Mensaje** — descripción de qué cambió y por qué.
- **Referencia al commit que le sigue** — así se forma la cadena de historia.

Esa cadena es clave: cada commit apunta al que lo precedió, formando una línea de tiempo que podés recorrer hacia atrás en cualquier momento.

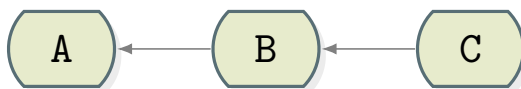


Figura 2.1: Línea de tiempo de commits

En la Figura 2.1 se muestra una secuencia de commits A, B y C, donde cada uno apunta al anterior, formando una historia lineal del proyecto.

Un commit es como el botón de guardar partida en un videojuego. No solo guardás el progreso actual, sino que podés tener múltiples puntos de guardado y volver exactamente a cualquiera de ellos si algo sale mal.

Git identifica cada commit con un hash SHA-1, un código único como `a3f4c91...`. Ese hash es calculado a partir del contenido del commit, de este modo git garantiza que el contenido no fue corrompido o alterado, lo que hace a los commits confiables e inmutables.

El mensaje o **commit message** debe describir de forma clara qué se cambió y por qué. Buenas prácticas recomendadas:

- usar frases breves y concretas;

- escribir en imperativo (por ejemplo: “Agrega README.md”);
- evitar mensajes genéricos como “cambios” o “arreglos”;
- si hace falta, agregar una descripción más larga explicando contexto.

2.3. Áreas de un proyecto Git

El flujo de trabajo de Git se organiza en tres áreas principales: el **working directory** (area de trabajo), donde se realizan las modificaciones; el **staging area** (area de preparación), donde se seleccionan los cambios que formarán parte del próximo commit; y el **local repository** (repositorio local), donde se almacenan de forma permanente las instantáneas confirmadas. Este enfoque permite un control preciso sobre qué cambios se registran en cada commit.

En resumen, el flujo de trabajo típico en Git implica:

- **working directory**: directorio de trabajo donde se crean, editan o eliminan archivos;
- **staging area**: área intermedia donde se seleccionan de forma explícita los cambios que formarán parte del próximo commit;
- **local repository**: repositorio local almacenado en `.git`, donde Git guarda el historial completo del proyecto.

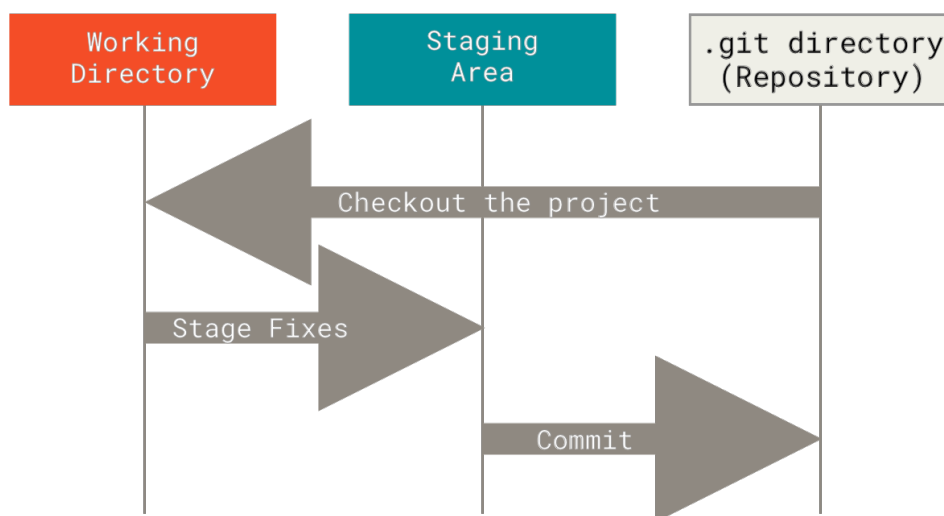


Figura 2.2: Flujo de trabajo en Git

2.4. Branch

Una branch (o rama) en Git es una **línea de tiempo paralela** dentro de tu proyecto. Las ramas son utilizadas para desarrollar funcionalidades aisladas unas de otras. La rama **main** o **master** es la rama “por defecto” cuando creas un repositorio, y es donde se encuentra la **versión estable del código**.

Durante el desarrollo, es común crear ramas para trabajar en nuevas características, corregir errores o experimentar sin afectar la rama principal. Este enfoque es útil a la hora

de trabajar colaborativamente, ya que cada desarrollador puede tener su propia rama para sus cambios, evitando modificar el código sobre el que alguien más está trabajando.

Cada rama es esencialmente una **referencia o puntero** a un commit específico en la historia del proyecto. Cuando haces un commit en esa rama, el puntero se mueve hacia adelante, estando **siempre apuntando al último commit de esa línea de desarrollo**.

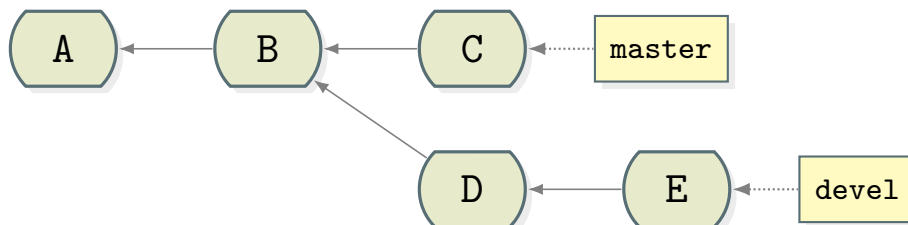


Figura 2.3: Ejemplo de ramas en Git

En la figura 2.3 se muestra una rama principal (master) que avanza con los commits A, B y C, mientras que una rama de desarrollo (devel) se bifurca en D y E. Cada rama apunta al último commit de su línea de desarrollo.

2.5. Puntero HEAD

Git utiliza un puntero llamado HEAD para **indicar en qué lugar del árbol estás parado actualmente**. Normalmente HEAD apunta a la rama activa, lo que significa que cualquier commit que hagas se agregará a esa rama.

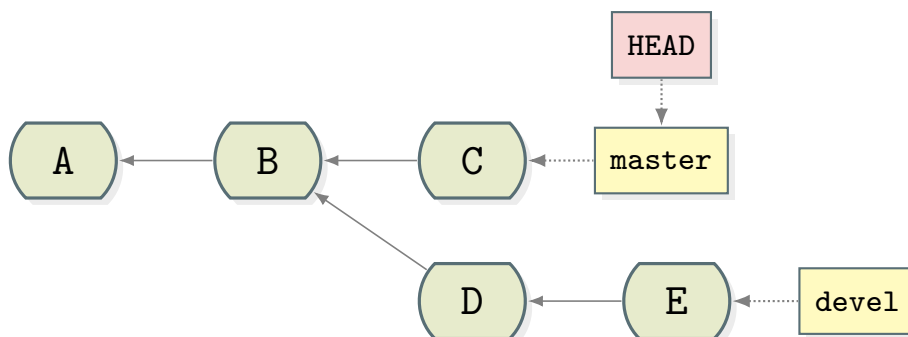


Figura 2.4: Puntero HEAD apuntando a la rama master

En la figura 2.4 se muestra el puntero HEAD apuntando a la rama master, lo que indica que estamos trabajando en esa rama. Si hacemos un commit, se agregará después de C, avanzando la rama master.

Es posible apuntar HEAD a un commit específico, este estado se llama "detached HEAD" y es útil para revisar versiones anteriores, pero no nos permite hacer commits permanentes sin crear una nueva rama.

En la figura 2.5 se muestra el puntero HEAD apuntando al commit B, lo que indica que estamos en un estado de detached HEAD. En este estado no deben hacerse commits, ya que no estarán asociados a ninguna rama y podrían perderse fácilmente.

Veremos que el comando `git checkout` se usa para cambiar de rama o para moverse a un commit específico, lo que no es mas que mover el puntero HEAD.

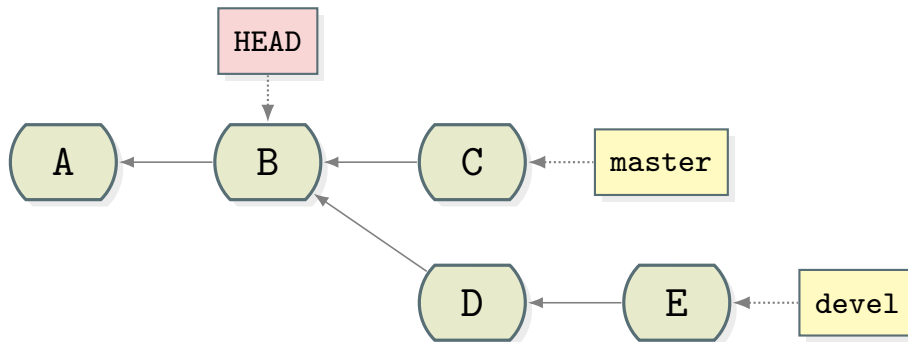


Figura 2.5: Puntero HEAD apuntando al commit B (detached HEAD)

2.6. Merge

Merge significa unir dos ramas. A la hora de hacer un merge, Git toma los cambios de una rama (la rama fuente) y los integra en otra rama (la rama destino). En otras palabras, hacer un merge es traer los cambios de una rama fuente a una rama de destino. Git tomará la rama actual (HEAD) como destino. Existen dos formas de hacer un merge:

- **Fast-forward** (avance rápido): Si la rama de destino no tiene nuevos commits desde que se creó la rama fuente (las ramas no se han separado), Git simplemente mueve el puntero de la rama de destino hacia adelante, apuntando al mismo commit que la rama fuente. En este caso, no se crea un nuevo commit de merge y la historia es se mantiene lineal.

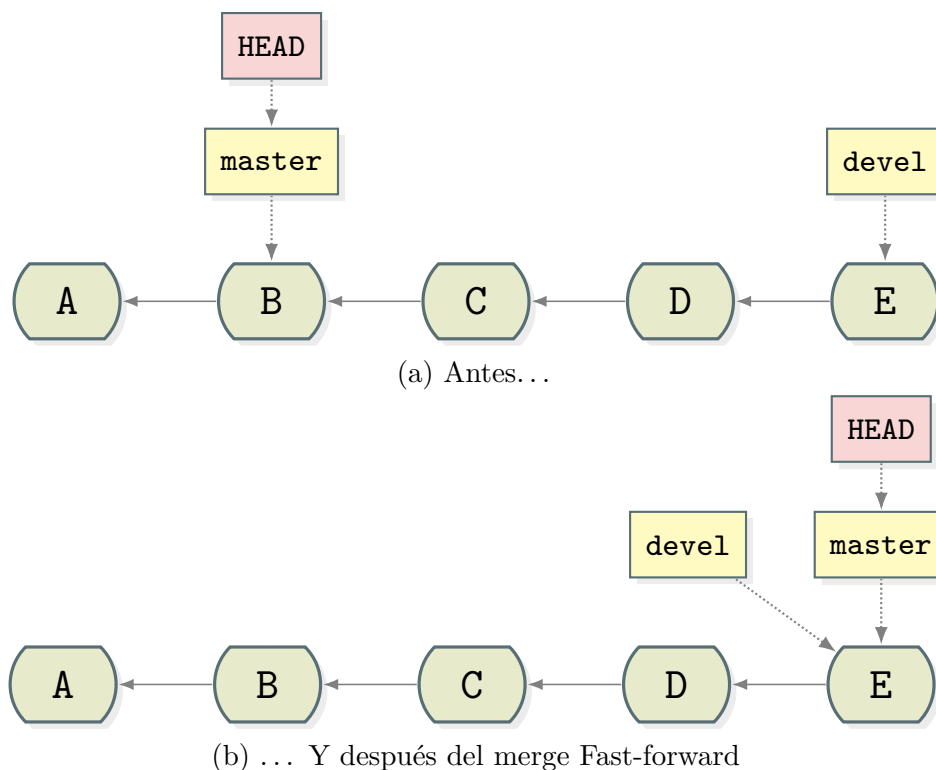


Figura 2.6: Merge Fast-forward

En la figura 2.6 se muestra un merge desde la rama devel hacia master. Antes del merge, master apunta a B y devel apunta a E. Después del merge Fast-forward,

master también apunta a E, avanzando su puntero sin crear un nuevo commit de merge.

- **No-fast-forward** (sin avance rápido): Si ambas ramas avanzaron por separado, Git crea un nuevo commit especial que tiene dos "padres" el último commit de cada rama. Es la unión real de las dos historias.

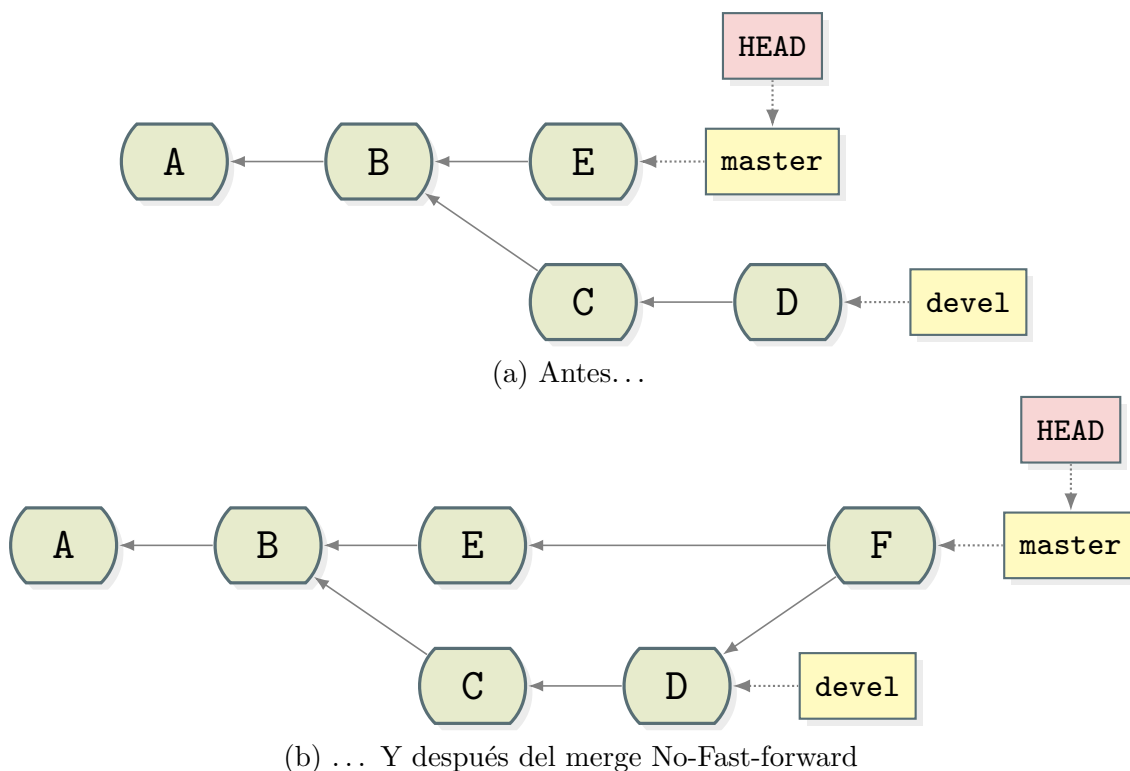


Figura 2.7: Merge No-Fast-forward

En la figura 2.7 se muestra un merge desde la rama devel hacia master. Antes del merge, master apunta a E y devel apunta a D. Después del merge No-Fast-forward, master apunta a un nuevo commit F que tiene dos padres: E (último commit de master) y D (último commit de devel). Este nuevo commit F representa la unión de las dos ramas.

Por defecto, Git realizará un merge fast-forward siempre que sea posible.

Al realizar un merge, es posible que surja un conflicto: esto se da si las dos ramas modificaron el mismo fragmento del mismo archivo de formas distintas, Git no sabe cuál versión conservar y te pide que lo resuelvas a mano. Ya sea eligiendo una de las dos versiones, o combinándolas.

2.7. Push y Pull

A la hora de trabajar con repositorios remotos, los comandos `git push` y `git pull` son fundamentales para sincronizar tu trabajo local con el servidor remoto.

Hacer un **push** significa enviar tus commits locales a un repositorio remoto, actualizando la rama correspondiente en el servidor. Es **subir tu trabajo** para compartirlo con otros o para tener un respaldo en la nube.

Hacer un **pull** significa traer los cambios del repositorio remoto a tu máquina local, e integrarlos (merge) con tu trabajo actual. Es **descargar el trabajo de otros** y mezclarlos con tu trabajo local para mantener tu copia actualizada.

2.8. Pull Request

En github, **pull request** (PR) es una solicitud para que tus cambios sean revisados e integrados en otra rama de un proyecto. Es una forma de colaborar en proyectos compartidos, permitiendo que otros revisen tu código, hagan comentarios y sugerencias antes de que tus cambios se fusionen con la rama principal.

Es básicamente la **solicitud de merge** de la rama donde está tu trabajo hacia la otra rama del proyecto, pero en el repositorio remoto.

Usando Git en la práctica

Aprenderemos git desde la terminal ya que, aunque existen interfaces gráficas que pueden facilitar su uso, conociendo los conceptos básicos y comandos fundamentales, es muy sencillo después usar cualquiera de esas interfaces gráficas, ya que todas se utilizan estos mismos comandos por debajo.

3.1. Instalación

En sistemas basados en Debian/Ubuntu:

```
$ sudo apt update
$ sudo apt install git
```

Podemos comprobar la instalación con:

```
$ git --version
git version 2.53.0
```

3.2. Configuración inicial

Antes de empezar a usar Git, es importante configurar tu identidad para que los commits tengan un autor asociado. Esto se hace con los siguientes comandos:

```
$ git config --global user.name "Tu Nombre"
$ git config --global user.email "Tu email"
```

Podemos comprobar la configuración con:

```
$ git config --list
user.name=Tu Nombre
user.email=Tu email
```

3.3. Como obtener ayuda

Es de utilidad conocer los comandos de ayuda de Git para resolver dudas o aprender sobre nuevas funcionalidades. Algunos comandos útiles son:

```
$ git help <comando> # Muestra la ayuda detallada de un comando específico
$ git <comando> --help # Otra forma de mostrar la ayuda de un comando
$ git help # Muestra una lista de comandos disponibles y su descripción breve
```

3.4. Creando un repositorio vacío

Para crear nuestro primer repositorio, primero debemos crear un directorio para nuestro proyecto y luego inicializar un repositorio Git dentro de ese directorio:

```
$ mkdir mi-proyecto
$ cd mi-proyecto
$ git init
Initialized empty Git repository in ../mi-proyecto/.git/
```

Esto crea un nuevo repositorio Git vacío en el directorio mi-proyecto. Ahora podemos empezar a agregar archivos, hacer cambios y registrar esos cambios con commits.

Para comprobar que el repositorio se creó correctamente, podemos usar el comando `git status`:

```
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

3.5. Realizando mi primer commit

A modo de ejemplo, podemos crear un archivo `README.md` y agregarlo al repositorio:

```
$ echo "# Mi Proyecto" > README.md
```

Si ejecutamos `git status`, veremos que el archivo `README.md` no está siendo rastreado por Git:

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Para agregar el archivo al área de preparación (staging area), usamos el comando `git add`:

```
$ git add README.md
```

Ahora el archivo README.md está listo para ser incluido en el próximo commit. Podemos verificarlo con `git status`:

```
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
```

Finalmente, para registrar el cambio en el historial del repositorio, hacemos un commit con el comando `git commit`:

```
$ git commit -m "Agrega README.md"
[main (root-commit) d37249c] Agraga README.md
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

Luego de hacer el commit, podemos ver el historial de commits con el comando `git log`:

```
$ git log
commit d37249c965d8ccadb4aca126435412ae11314233 (HEAD -> main)
Author: Tu Nombre <tu email>
Date:   Wed Apr 29 01:40:03 2026 -0300

    Agraga README.md
```

En el historial de commits se muestra el hash del commit, el autor, la fecha y el mensaje del commit. También podemos observar que el puntero HEAD apunta a main, por lo que estamos trabajando en dicha rama. Y main, a su vez, apunta al commit d37249c, que es el commit que acabamos de hacer.

3.6. Subir repositorio a GitHub

Para poder autenticarte con GitHub desde tu computadora, es recomendable generar una clave SSH y agregarla a tu cuenta de GitHub. Esto te permitirá realizar operaciones como push y pull sin tener que ingresar tus credenciales cada vez. Para esto, puedes usar el siguiente comando para generar una nueva clave SSH:

```
$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/corte/.ssh/id_ed25519):
```

```
Created directory '/home/corte/.ssh'.
Enter passphrase for "/home/corte/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/corte/.ssh/id_ed25519
Your public key has been saved in /home/corte/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:HyLPaploiQfNDg3sallUeyaLr5ZX5J7eDUxYJYwNQg corte@Corte-ThinkPad
The key's randomart image is:
+--[ED25519 256]--+
|  Eoo+o.          |
|  ....=.         |
|  o.  o .         |
|  . *.o  o        |
|o.%o+ . S +       |
|+Xoo ..+ = .      |
|=... oo.+ o       |
|..+...++         |
|. =o  oo          |
+-----[SHA256]-----+
```

Damos tres veces a enter para aceptar las opciones por defecto y generar la clave sin passphrase. Luego, debemos agregar el contenido de la clave pública a nuestra cuenta de GitHub. Para esto, podemos usar el siguiente comando para mostrar el contenido de la clave pública:

```
$ cat ~/.ssh/id_ed25519.pub    #Prestar atención a la ruta del archivo, debe
→ ser la misma que se mostró al generar la clave.
ssh-ed25519
→ AAAAC3NzaC1lZDI1NTE5AAAAHyLPaploiQfNDg3sallUeyaLr5ZX5J7eDUxYJYwNQg
→ corte@Corte-ThinkPad
```

Copiamos el contenido del archivo, y vamos a GitHub→Settings→SSH and GPG keys→New SSH key, pegamos el contenido y guardamos la nueva clave SSH.

Si tienen problemas con esta parte, les dejo un video explicativo: https://www.youtube.com/watch?v=_vlrPpxs2Go

3.7. workflow